

Research topics

Topic names only, no solutions, for when you're stuck. Numbers in brackets point to stories (1.1) and open decisions (D1) on the requirements tab.

1 · Auth & authorization

How users prove who they are, and how the server decides what each of them may touch.

- Session cookies vs token-based auth (JWTs), and their trade-offs (1.2)
- Password hashing algorithms and work factors (1.2)
- Single-use, expiring tokens for email verification and resets (1.1, 1.2)
- Account enumeration (1.1)
- Session management: fixation, rotation, idle timeout, revocation (1.2)
- Role-based vs attribute-based access control (RBAC vs ABAC) (1.3–1.6)
- Broken object-level authorization (BOLA, also called IDOR) (1.3, 1.4)
- Returning 404 vs 403 for resources a user may not see (1.3)
- Invitation flows and time-limited, scoped permissions (1.5)
- Audit logging (1.6)
- Step-up authentication (1.7)

2 · Domain & data rules

How to keep the inventory rule true when many requests hit the same data at once. The transactions chapter of *Designing Data-Intensive Applications* covers most of this list.

- Race conditions and check-then-act bugs (2.2, 2.4, 2.5)
- Transaction isolation levels: read committed, repeatable read, serializable (2.2, D5)
- Lost updates, write skew and phantoms (2.2, 2.4)
- Pessimistic locking vs optimistic concurrency control (2.2, D5)
- Atomic conditional updates (compare-and-set) (2.2, 2.5)
- Database constraints as enforced invariants (2.2, 2.5)
- Reservations with expiry (2.2, 3.2)
- Finite state machines for order lifecycles (2.3)
- Append-only audit logs vs event sourcing (2.3, 2.6)

- Representing money: minor units, decimal types, floating-point pitfalls (2.1)
- Times across time zones: IANA zone names, daylight-saving edge cases (2.1)
- Schema migrations and backward-compatible changes (2.1, 6.3)

3 · Async work

How work that happens later stays correct when processes crash, messages repeat or events arrive out of order.

- Job queues, workers, and job leases or visibility timeouts (3.1–3.6)
- Delivery guarantees: at-most-once, at-least-once, "exactly-once" (3.3, 3.5)
- Idempotency and idempotency keys (3.3, 3.5, 4.3)
- The dual-write problem and the transactional outbox pattern (3.3, 3.5)
- Retries with exponential backoff and jitter (3.5, 3.6)
- Dead-letter queues (3.5, 3.6)
- Durable timers and scheduled jobs (3.2, 3.4)
- Webhook delivery semantics: retries, duplicates, ordering (3.3)
- Token bucket and leaky bucket algorithms for pacing (3.1)
- Virtual waiting rooms (3.1, D2)
- Queue fairness: first-come-first-served vs randomized admission (D2)
- Payment reconciliation (3.4)
- Sagas and compensating actions (3.6, D1)

4 · Frontend state

How the browser shows the server's truth in real time and stays honest when the network isn't.

- Server state vs UI state (4.1–4.6)
- Caching and stale-while-revalidate (4.1)
- Polling vs long polling vs Server-Sent Events vs WebSockets (4.1, 4.2, 4.7, D4)
- Reconnection with backoff, and online/offline detection (4.5)
- Clock skew and server-authoritative countdowns (4.3)
- Statecharts and finite state machines for UI flows (4.2–4.4)
- Optimistic vs pessimistic UI updates (4.3, 4.4)

- Preventing double submission (4.3)
- Cross-tab synchronization (4.2)
- Loading skeletons, empty states and error boundaries (4.1, 4.6)
- ARIA live regions and focus management (4.9)
- WCAG 2.2 success criteria and automated accessibility testing (4.9, 8.7)
- Core Web Vitals: LCP, INP, CLS (4.10)
- Code splitting and performance budgets (4.10)
- Camera access and QR decoding in the browser (4.8)

5 • Security

How to stop bots, forgers and curious users without hurting real buyers.

- Rate limiting algorithms: fixed window, sliding window, token bucket (5.1, 5.4, 5.7)
- Bot mitigation and anti-scalping measures, and their trade-offs (5.1)
- Digital signatures vs HMAC (5.2, 5.3, D6)
- Unguessable identifiers and entropy (1.4, 5.2)
- Webhook signature verification and replay protection (5.3)
- PCI DSS scope and hosted payment pages (5.3)
- Trust boundaries and server-side input validation with schemas (5.3, 5.6)
- File upload security: magic bytes, polyglot files, image re-encoding, metadata stripping (5.4)
- Serving user uploads safely: content types, Content-Disposition, separate domains (5.4)
- Cross-site scripting (XSS), output encoding and Content Security Policy (5.6)
- CSV and formula injection (5.5)
- Cross-site request forgery (CSRF) and SameSite cookies (1.2, 5.6)
- Brute-force protection vs lockout abuse (5.7)
- Breached-password checks with k-anonymity (5.7)
- GDPR basics: data minimization, erasure vs retention, anonymization vs pseudonymization (5.8)
- OWASP ASVS and the OWASP Top 10 as checklists (all)

6 · Shipping

How to get the system running, deployed, observed and recoverable without manual steps.

- Container images and multi-stage builds (6.1)
- Local environments with Docker Compose and seed data (6.1)
- CI pipelines: caching, parallel jobs, branch protection (6.2)
- Deployment strategies: rolling, blue-green, canary (6.3)
- Liveness vs readiness health checks (6.3)
- Automated rollback (6.3)
- Zero-downtime migrations with expand and contract (6.3)
- Structured logging and correlation IDs (6.5)
- The RED and USE methods for metrics (6.5)
- Distributed tracing and OpenTelemetry (6.5)
- SLOs, error budgets, and alerting on symptoms (6.5)
- Error tracking and scrubbing personal data (6.5)
- RPO, RTO, point-in-time recovery and restore drills (6.6)
- Secrets management and secret scanning (6.7)
- Twelve-Factor App configuration (6.7)
- Cloud cost controls and spend alerts (6.8)

7 · Documentation

How to make your decisions, API and results easy for a reviewer to find and trust.

- Architecture Decision Records and Michael Nygard's format (7.2)
- The C4 model for architecture diagrams (7.1)
- Diagrams as code, such as Mermaid (7.1)
- README structure for open-source projects (7.1)
- OpenAPI, and keeping a spec in sync with the code (7.3)
- Latency percentiles vs averages (7.4)
- Runbooks and incident response basics (7.5)
- Conventional Commits, semantic versioning and Keep a Changelog (7.6)

8 · Tests

How to prove the risky rules hold, including under concurrency, time pressure and load.

- The test pyramid vs the testing trophy (all)
- Testing concurrent code: parallel requests, barriers, repeated runs (8.1)
- Integration tests against a real database in containers (8.1–8.3)
- Property-based testing (8.1, 8.2)
- Injectable clocks for time-based tests (8.4)
- Testing third-party integrations: sandbox modes, recorded webhooks, contract tests (8.2)
- Table-driven tests for access-control matrices (8.3)
- End-to-end browser testing and controlling flaky tests (8.5)
- Load testing: open vs closed workload models, ramp-up, coordinated omission (8.6)
- Fault injection, such as killing workers mid-job (3.2, 3.5, 8.2)
- Security testing: DAST scanners and fuzzing (8.8)